

Python Package Setup: Shared common Module

Project Structure

```
Anthropic-Courses/  
├── .env  
├── conftest.py      ← pytest path fix  
├── common/  
│   ├── __init__.py  ← makes common a package  
│   └── env.py        ← load_environment_variables()  
├── tests/  
│   └── test_env.py   ← run pytest from Anthropic-Courses/  
└── BuildingWithClaudeAPI/  
    └── APIRequests/  
        └── my_script.py
```

Step 1 — Make **common** a Python package

Create an empty `__init__.py` inside `common/`:

```
touch Anthropic-Courses/common/__init__.py
```

This tells Python that `common` is a package, enabling imports like:

```
from common.env import load_environment_variables
```

Step 2 — Fix pytest's path with `conftest.py`

Create `Anthropic-Courses/conftest.py`:

```
import sys  
from pathlib import Path
```

```
sys.path.insert(0, str(Path(__file__).parent))
```

pytest automatically loads every `conftest.py` it finds on the way to your test files. This one adds `Anthropic-Courses/` to `sys.path`, so any test file anywhere in the tree can do `from common.env import ...` without extra setup.

Without this file, Python would not know where `common` lives and every test would fail with `ModuleNotFoundError: No module named 'common'`.

This file only affects pytest runs. It has no effect when running scripts directly.

Step 3 — Put tests in `Anthropic-Courses/tests/`

Keep tests under `Anthropic-Courses/tests/` and always run pytest from `Anthropic-Courses/`:

```
cd Anthropic-Courses
pytest tests/test_env.py -v
```

This matters for `.env` discovery. `env.py` uses:

```
find_dotenv(usecwd=True)
```

`find_dotenv` searches **upward** from the current working directory. Running from `Anthropic-Courses/` means it finds `.env` immediately. Running from the repo root would search upward and never find it (it is in a subdirectory, not a parent).

Step 4 — Two kinds of tests: mocked vs real

Mocked — use when testing edge cases (not found, specific values):

```
def test_load_environment_variables_not_found(capsys):
    with patch('common.env.find_dotenv', return_value=None):
        result = load_environment_variables()
        assert result is None
```

Real — use when you want to verify against the actual `.env` file. No mocking needed; just run from `Anthropic-Courses/` and `find_dotenv` handles discovery automatically:

```
def test_read_prod_githubname():
    load_environment_variables()
    assert os.getenv("GITHUB_NAME") == "sunitghub"
```

No path guessing, no monkeypatch.chdir — the directory placement does the work.

Step 5 — Running scripts directly (not via pytest)

`conftest.py` is pytest-only. For scripts run directly, set `PYTHONPATH` in your shell before running:

```
export PYTHONPATH=/path/to/Anthropic-Courses
python BuildingWithClaudeAPI/APIRequests/my_script.py
```

Add this export to your shell profile (`~/.zshrc` or `~/.bashrc`) to make it permanent, or activate it per-session.

Why not `.env`? Python reads `PYTHONPATH` at startup, before any Python code runs — so `load_dotenv()` is too late to affect it. The `.env` file works for API keys and other runtime variables, but not for `PYTHONPATH`.

Step 6 — The `env.py` module

`common/env.py` provides `load_environment_variables()`, which:

1. Searches for a `.env` file starting from the current working directory upward
2. Loads it with `python-dotenv`
3. Returns the directory containing the `.env` file (useful for building other paths)

Usage from any script:

```
from common.env import load_environment_variables

root = load_environment_variables()
# root is the Path to the directory containing .env
```

Summary of files

File	Purpose
common/__init__.py	Makes common a Python package
conftest.py	Adds Anthropic-Courses/ to sys.path for all pytest runs
tests/test_env.py	Tests live here; run pytest from Anthropic-Courses/
common/env.py	load_environment_variables() — discovers and loads .env
.env	Secrets and config; documents PYTHONPATH for direct script runs